UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación · Carrera de Software

Taller de revisión técnica del código del PFC: principios SOLID y patrones de diseño

FORM-TL-06 · Guía para el estudiante y lista de verificación

Aplicaciones Distribuidas (ISR-701) · SPA 2026–2027

Campo	Detalle
Código	FORM-TL-06
Modalidad	Grupal, equipos del PFC de 3 o 4 integrantes
Carácter	Formativa: no tiene calificación, pero es requisito para presentar las sumativas que prepara
Prepara	GA-SUM-07 PE-U5 CI/CD, pruebas y observabilidad, y TA-PFC-E4 entrega final del PFC (semana 17)
Semana	15 (1 al 7 de febrero de 2027; el lunes 1 es feriado)
Fechas clave	Revisión del equipo e *issues*: sábado 6 de febrero de 2027, 23:59. Revisión del docente: semana 16. Resolución de los *issues*: domingo 14 de febrero de 2027, 23:59
Material de apoyo	Guía *Desarrollo de Aplicaciones Distribuidas por Capas*, capítulos 2 y 11
Docente	Gleiston C. Guerrero-Ulloa, PhD (`gguerrero@uteq.edu.ec`)

1. Propósito del taller

Antes de la entrega final, el equipo revisa su propio código con una lista de verificación de calidad, convierte cada problema encontrado en una tarea concreta con responsable y fecha, y recibe la revisión del docente sobre el mismo código. La revisión de código entre pares no solo detecta defectos: también difunde el conocimiento del sistema en el equipo y mejora la comprensión del código de los demás [1], y los componentes que reciben una revisión con poca participación tienden a acumular más defectos después de publicados [2].

Qué se espera al final de la semana 16

Una lista de verificación completa con evidencia en cada ítem, un *issue* de GitHub por cada mejora identificada, asignado a un integrante y con fecha de resolución, y las mejoras resueltas mediante *pull requests* antes de la semana 17. En la práctica, el docente debe poder abrir cualquier ítem de la lista y llegar, con un clic, a las líneas de código que lo respaldan.

2. Cómo se desarrolla el taller

Momento	Qué hace el equipo
Antes de la sesión	Etiqueta el estado actual del código con `tl06-base`, crea las etiquetas de *issue*, el hito con fecha y la plantilla de *issue* (sección 4). Cada integrante lee la lista de verificación (sección 3).
Sesión de laboratorio	Revisión en parejas: cada integrante revisa código que **no escribió**. Cada pareja recorre las cinco categorías sobre los servicios que le corresponden y anota cada ítem con su estado y su evidencia.
Hasta el sábado 6 de febrero	El equipo consolida la lista, redacta el documento de revisión (sección 5) y crea un *issue* por cada mejora, asignado y en el hito.
Semana 16	El docente revisa el repositorio en la etiqueta `tl06-base` con la misma lista y comenta en GitHub. Cada comentario del docente que pida un cambio se convierte en un *issue* nuevo.
Hasta el domingo 14 de febrero	Cada responsable resuelve sus *issues* con un *pull request* que los cierra, revisado por otro integrante.

3. Lista de verificación

Cada ítem se marca como **Cumple**, **Mejora** (no cumple y se abre un *issue*) o **No aplica** (con justificación). Todo ítem marcado como Cumple o Mejora lleva como evidencia un enlace permanente (*permalink*) a las líneas de código en la etiqueta `tl06-base`; en GitHub se obtiene seleccionando las líneas y eligiendo *Copy permalink*.

A. Principios SOLID (al menos tres principios verificados con evidencia)

Los principios SOLID buscan que los módulos toleren el cambio, sean fáciles de entender y puedan reutilizarse [3].

ID	Ítem verificable	Cómo se verifica
A1	Responsabilidad única: cada controlador solo traduce HTTP y cada servicio atiende un solo grupo de casos de uso.	Buscar en los controladores llamadas a repositorios, cálculos de negocio o SQL. Clases de más de 300 líneas o con más de cinco dependencias son candidatas a dividirse.
A2	Abierto/cerrado: un comportamiento nuevo se agrega sin modificar el código que ya funciona.	Buscar cadenas largas de `if` o `switch` sobre un tipo (por ejemplo, método de pago o tipo de usuario) que habría que editar para cada caso nuevo.
A3	Sustitución de Liskov: las subclases e implementaciones respetan el contrato de su tipo base.	Buscar implementaciones que lancen `UnsupportedOperationException` o que ignoren parámetros del contrato.
A4	Segregación de interfaces: ninguna clase depende de métodos que no usa.	Interfaces con muchos métodos de los que cada cliente usa solo algunos.

ID	Ítem verificable	Cómo se verifica
A5	Inversión de dependencias: las clases reciben sus dependencias por el constructor y dependen de abstracciones.	Buscar `new` de servicios, repositorios o clientes HTTP dentro de otras clases, e inyección por campo (`@Autowired` sobre atributos). Revisar los antipatrones de inyección catalogados en [4].

B. Patrones de diseño aplicados correctamente

Un patrón es una solución probada a un problema que se repite [5]. El equipo identifica **al menos tres** patrones presentes en el PFC y verifica que estén bien aplicados.

ID	Ítem verificable	Cómo se verifica
B1	Cada patrón declarado existe en el código y resuelve un problema real del PFC.	Para cada patrón: clase donde se aplica, problema que resuelve y qué pasaría sin él.
B2	El repositorio (*Repository*) no filtra detalles de la base de datos a la capa de negocio [6].	Buscar en los servicios SQL escrito a mano, `EntityManager` o tipos propios del controlador de la base de datos.
B3	Las fábricas, estrategias u observadores tienen una sola forma de uso y no se saltan.	Buscar objetos creados con `new` que deberían crearse con la fábrica, o eventos que se notifican llamando directamente al receptor.
B4	No hay «patrones de nombre»: clases llamadas `Factory`, `Strategy` o `Manager` que no implementan el patrón.	Comparar el nombre de la clase con su estructura.

C. Separación de capas clara

Las capas solo deben usar a la capa inmediatamente inferior; los atajos erosionan la arquitectura con el tiempo [7], [8].

ID	Ítem verificable	Cómo se verifica
C1	Ninguna clase de negocio o de dominio importa clases de la capa web.	Buscar en los paquetes de negocio importaciones de paquetes web, de anotaciones HTTP o de `ResponseEntity`.
C2	Los controladores no usan repositorios directamente.	Buscar repositorios inyectados en controladores.
C3	El API recibe y devuelve DTO, no entidades de la base de datos.	Revisar las firmas públicas de los controladores.
C4	Las transacciones se delimitan en la capa de negocio.	Buscar `@Transactional` en controladores o su ausencia en operaciones que modifican varias tablas.

ID	Ítem verificable	Cómo se verifica
C5	Las reglas de capas están automatizadas.	Existe al menos una prueba (por ejemplo, con ArchUnit) que falla si se viola C1 o C2.

D. Manejo de excepciones distribuidas

En un sistema distribuido una llamada remota puede fallar, tardar o devolver un error de otro servicio; el código debe tratar esos casos de forma explícita.

ID	Ítem verificable	Cómo se verifica
D1	Cada servicio tiene un manejador global de errores que responde con un formato común, como Problem Details (RFC 9457) [9].	Provocar un error de validación y uno de negocio y revisar la respuesta.
D2	Los códigos HTTP corresponden a la causa (400, 404, 409, 422, 503) y nunca se devuelve 200 con un error en el cuerpo.	Revisar la tabla de correspondencias del manejador.
D3	El cliente nunca recibe trazas internas, mensajes de la base de datos ni datos sensibles.	Provocar un error inesperado y revisar la respuesta.
D4	Toda llamada a otro servicio o al *broker* tiene tiempo de espera, y los reintentos solo se aplican a operaciones idempotentes.	Revisar la configuración de cada cliente HTTP, gRPC o de mensajería.
D5	La caída de un servicio dependiente produce una respuesta controlada, no la caída en cascada del que llama.	Detener el servicio dependiente con Docker Compose y observar al que llama.
D6	No hay bloques `catch` vacíos ni excepciones capturadas y descartadas sin registro.	Buscar `catch` con cuerpo vacío o con solo un comentario.

E. Logging y trazabilidad

Cuando un sistema distribuido falla, solo se puede entender lo ocurrido a partir de lo que registra sobre sí mismo [10].

ID	Ítem verificable	Cómo se verifica
E1	Se usa un *framework* de registro con niveles (no `System.out` ni `print`).	Buscar `System.out`, `printStackTrace` o `print(` en el código de producción.

ID	Ítem verificable	Cómo se verifica
E2	Los registros tienen formato estructurado (por ejemplo, JSON) con fecha, nivel, servicio y mensaje.	Revisar la configuración de registro y una línea real del registro.
E3	Un identificador de traza o de correlación viaja entre servicios y aparece en los registros de todos ellos. Puede seguir el estándar W3C Trace Context (cabecera `traceparent`) [11].	Hacer una solicitud que atraviese al menos dos servicios y buscar el mismo identificador en los registros de ambos.
E4	Los eventos de negocio relevantes (creación, cambio de estado, rechazo) quedan registrados con su identificador.	Ejecutar un caso de uso y revisar el registro.
E5	Los registros no contienen contraseñas, tokens ni datos personales sensibles.	Buscar en los registros y en el código que los escribe.
E6	Cada servicio expone su estado de salud.	Consultar el punto de salud de cada servicio.

4. Issues en GitHub

4.1. Preparación del repositorio

El equipo crea las etiquetas por categoría y un hito (*milestone*) cuya fecha límite es el domingo 14 de febrero de 2027. GitHub no tiene un campo de fecha por *issue*: la fecha de resolución la da el hito al que se asigna. Con la herramienta de línea de comandos de GitHub, desde la carpeta del repositorio:

```
gh label create form-tl-06 --color 1E5AA8 --description "Taller de revision FORM-TL
    -06"
gh label create solid --color 0E3F7E
gh label create patrones --color 2E86A1
gh label create capas --color 4FAE60
gh label create excepciones --color B02418
gh label create logging --color F2A413
gh api repos/{owner}/{repo}/milestones -f title="FORM-TL-06 mejoras" -f due_on
    ="2027-02-14T23:59:59Z"
git tag -a tl06-base -m "Codigo revisado en FORM-TL-06"
git push origin tl06-base
```

4.2. Plantilla de issue

Para que todos los *issues* tengan la misma información, el equipo agrega una plantilla de formulario en `.github/ISSUE_TEMPLATE/mejora.yml` [12]:

```
name: Mejora de calidad (FORM-TL-06)
description: Registrar una mejora identificada en la revision tecnica del codigo del
```

```
    PFC.
title: "[TL06][CATEGORIA] Titulo breve de la mejora"
labels: ["form-tl-06"]
body:
  - type: dropdown
    id: categoria
    attributes:
      label: Categoria de la lista de verificacion
      options:
        - "A. Principios SOLID"
        - "B. Patrones de diseno"
        - "C. Separacion de capas"
        - "D. Manejo de excepciones distribuidas"
        - "E. Logging y trazabilidad"
    validations:
      required: true
  - type: input
    id: item
    attributes:
      label: Item de la lista de verificacion
      placeholder: "Ejemplo: C2"
    validations:
      required: true
  - type: textarea
    id: hallazgo
    attributes:
      label: Hallazgo
      description: Que se encontro y por que viola el item de la lista.
    validations:
      required: true
  - type: textarea
    id: evidencia
    attributes:
      label: Evidencia
      description: Enlace permanente (permalink) al archivo y a las lineas del codigo.
    validations:
      required: true
  - type: textarea
    id: propuesta
    attributes:
      label: Propuesta de mejora
    validations:
      required: true
  - type: textarea
    id: aceptacion
    attributes:
      label: Criterio de aceptacion
      description: Como se comprobara que la mejora quedo resuelta (prueba, regla de
    ArchUnit, captura).
    validations:
      required: true
```

4.3. Reglas de los issues

1. Un *issue* por cada ítem marcado como Mejora; si un ítem falla en varios servicios, un *issue* por servicio.

2. Título con el formato `[TL06][ID] descripción breve`, por ejemplo: *[TL06][C2] El controlador de reservas usa el repositorio directamente.*
3. Etiqueta `form-tl-06` y la de su categoría, un único responsable asignado y el hito `FORM-TL-06 mejoras`.
4. Cada integrante es responsable de al menos dos *issues*.
5. Se resuelve con un *pull request* cuya descripción incluye `Closes #n` y que revisa y aprueba otro integrante. El criterio de aceptación del *issue* se comprueba antes de aprobar.

4.4. Revisión del docente

En la semana 16 el docente revisa el código de la etiqueta `tl06-base` con la misma lista de verificación. Comenta en las líneas de código de los *commits* y de los *pull requests* abiertos, y deja un resumen en un *issue* titulado `[TL06] Revisión docente`. El equipo responde cada comentario y abre un *issue* por cada cambio solicitado, con las mismas reglas.

5. Entregables

1. **Documento de revisión** en LaTeX, en `docs/revision-tl06/` del repositorio del PFC, con su PDF compilado. Contiene: la lista de verificación completa con estado, evidencia (*permalink*) y número de *issue* por ítem; los tres principios SOLID y los tres patrones verificados con su explicación; la tabla de *issues* con su responsable; quién revisó qué código en las parejas; y la declaración de uso de inteligencia artificial.
2. **Etiqueta** `tl06-base`, **hito** y ***issues*** creados en GitHub hasta el sábado 6 de febrero de 2027 a las 23:59.
3. **Documento en el SGA**: cada integrante sube **solamente un documento PDF de UNA SOLA página** con los datos identificativos y la URL del repositorio escrita completa en una sola línea.
4. ***Pull requests*** que cierran los *issues*, integrados hasta el domingo 14 de febrero de 2027 a las 23:59.

6. Condiciones para considerar entregada la formativa

Esta actividad no tiene calificación. Sin embargo, es requisito para presentar GA-SUM-07 y TA-PFC-E4: el estudiante cuya formativa no se considere entregada no puede presentar esas sumativas y obtiene cero en ellas, salvo justificación aprobada (regla de las actividades formativas del encuadre).

Criterios de piso: si no se cumplen, la formativa se considera NO ENTREGADA

P1. PDF de una sola página con carátula y URL. El estudiante debe subir al SGA solamente un documento PDF de UNA SOLA página con los datos identificativos y, en una sola línea, la URL del repositorio donde está el trabajo. Si el PDF tiene más de una página, si se suben otros archivos en lugar de este PDF, si la URL no cabe en una sola línea o está rota, o si el repositorio no es accesible, la formativa se considera no entregada.

P2. PDF reproducible. Si el PDF del documento de revisión no se genera correctamente al clonar el repositorio y compilar el archivo `.tex`, la formativa se considera no entregada. Las instrucciones de compilación (compilador, orden de comandos, archivo principal y dependencias) deben estar en el `README.md`.

P3. Contenido mínimo. La lista de verificación cubre las cinco categorías con evidencia, hay al menos tres principios SOLID verificados y existe al menos un *issue* asignado a cada integrante, todo hasta el sábado 6 de febrero de 2027 a las 23:59.

P4. Integridad. La lista de verificación o los *issues* copiados de otro equipo o generados por IA generativa y presentados como propios, o evidencias que no corresponden al código, hacen que la formativa se considere no entregada.

7. Retroalimentación formativa

El docente devuelve, en el *issue* `[TL06] Revisión docente`, una valoración cualitativa de cada aspecto. No produce nota, pero anticipa cómo se evaluarán esos mismos aspectos en GA-SUM-07 y TA-PFC-E4.

Tabla 1: Escala de la retroalimentación formativa

Aspecto	Logrado	En proceso	No logrado
Profundidad de la revisión	Cada ítem tiene estado, evidencia en líneas concretas y, si aplica, su *issue*.	Faltan evidencias en algunos ítems.	Ítems marcados sin evidencia.
Coincidencia con la revisión docente	El docente no encuentra problemas importantes que el equipo haya marcado como Cumple.	El docente encuentra uno o dos.	El docente encuentra varios.
Calidad de los *issues*	Hallazgo, evidencia, propuesta y criterio de aceptación verificable.	Propuestas vagas o sin criterio de aceptación.	*Issues* genéricos o sin evidencia.
Reparto y cierre	Todos los integrantes resuelven sus *issues* con *pull requests* revisados antes de la semana 17.	Parte de los *issues* quedan abiertos.	Los *issues* no se resuelven o los resuelve una sola persona.

Lista de verificación antes del sábado 6 de febrero

- ☐ La etiqueta `tl06-base` está en GitHub.
- ☐ Las cinco categorías tienen todos sus ítems con estado y evidencia.
- ☐ Hay al menos tres principios SOLID y tres patrones verificados.
- ☐ Cada ítem marcado como Mejora tiene su *issue*, con etiqueta, responsable y hito.
- ☐ Cada integrante es responsable de al menos dos *issues*.
- ☐ El documento de revisión compila desde un clon limpio y está en `docs/revision-tl06/`.
- ☐ Cada integrante subió al SGA el PDF de una página con la URL en una sola línea.

Referencias

[1] A. Bacchelli y C. Bird, «Expectations, Outcomes, and Challenges of Modern Code Review,» en *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA: IEEE, 2013, págs. 712-721. DOI: 10.1109/ICSE.2013.6606617.

[2] S. McIntosh, Y. Kamei, B. Adams y A. E. Hassan, «An Empirical Study of the Impact of Modern Code Review Practices on Software Quality,» *Empirical Software Engineering*, vol. 21, n.º 5, págs. 2146-2189, 2016. DOI: 10.1007/s10664-015-9381-9.

[3] R. C. Martin, *Clean Architecture, A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 978-0-13-449416-6.

[4] R. Laigner, D. Mendonça, A. Garcia y M. Kalinowski, «Cataloging Dependency Injection Anti-Patterns in Software Systems,» *Journal of Systems and Software*, vol. 184, pág. 111 125, 2022. DOI: 10.1016/j.jss.2021.111125.

[5] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns, Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, ISBN: 978-0-201-63361-0.

[6] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, ISBN: 978-0-321-12742-6.

[7] M. Richards y N. Ford, *Fundamentals of Software Architecture, A Modern Engineering Approach*, 2.ª ed. O'Reilly Media, 2025, ISBN: 978-1-098-17551-1.

[8] R. Li, P. Liang, M. Soliman y P. Avgeriou, «Understanding Software Architecture Erosion: A Systematic Mapping Study,» *Journal of Software: Evolution and Process*, vol. 34, n.º 3, e2423, 2022. DOI: 10.1002/smr.2423.

[9] M. Nottingham, E. Wilde y S. Dalal, «Problem Details for HTTP APIs,» RFC Editor, RFC 9457, jul. de 2023. DOI: 10.17487/RFC9457. dirección: https://www.rfc-editor.org/rfc/rfc9457.

[10] S. Niedermaier, F. Koetter, A. Freymann y S. Wagner, «On Observability and Monitoring of Distributed Systems – An Industry Interview Study,» en *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings*, S. Yangui, I. Bouassida Rodriguez, K. Drira y Z. Tari, eds., ép. Lecture Notes in Computer Science, vol. 11895, Cham: Springer, 2019, págs. 36-52. DOI: 10.1007/978-3-030-33702-5_3.

[11] S. Kanzhelev, M. McLean, A. Reitbauer, B. Drutu, N. Molnar e Y. Shkuro. «Trace Context.» W3C Recommendation, World Wide Web Consortium. (23 de nov. de 2021), dirección: https://www.w3.org/TR/2021/REC-trace-context-1-20211123/ (visitado 26-10-2026).

[12] GitHub. «Syntax for Issue Forms.» (2026), dirección: https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/syntax-for-issue-forms (visitado 26-10-2026).